CS4500 Operating Systems I
Project 2
POSIX Thread Programming

Name: ***Abe Anderson***
Date of Submission: ***date***

Statement: "I have neither given nor received unauthorized assistance on this work and am in compliance with Student Academic Ethics Code Policy."

AI usage disclosure: I have used Gen AI tool as a learning assistant in this work for scripting the testing document to run the program with several k and #threads for multiple runs
Name of the AI tool: GPT 4.0
Link of the Conversation: https://chatgpt.com/c/69c6bc96-8de0-83e8-9861-b682312f55c0

Task 1

1.1 How I Solved the Problems

To solve this problem, I used 4 threads split up by the work into 4 chunks, then each thread checks for substrings in its range. I also used a mutex so the shared total_count var doesn't get overridden. After each thread completes, it locks and updates the total_count, then unlocks so another thread can update. After all threads are complete, the total count is printed

1.2 What I learned

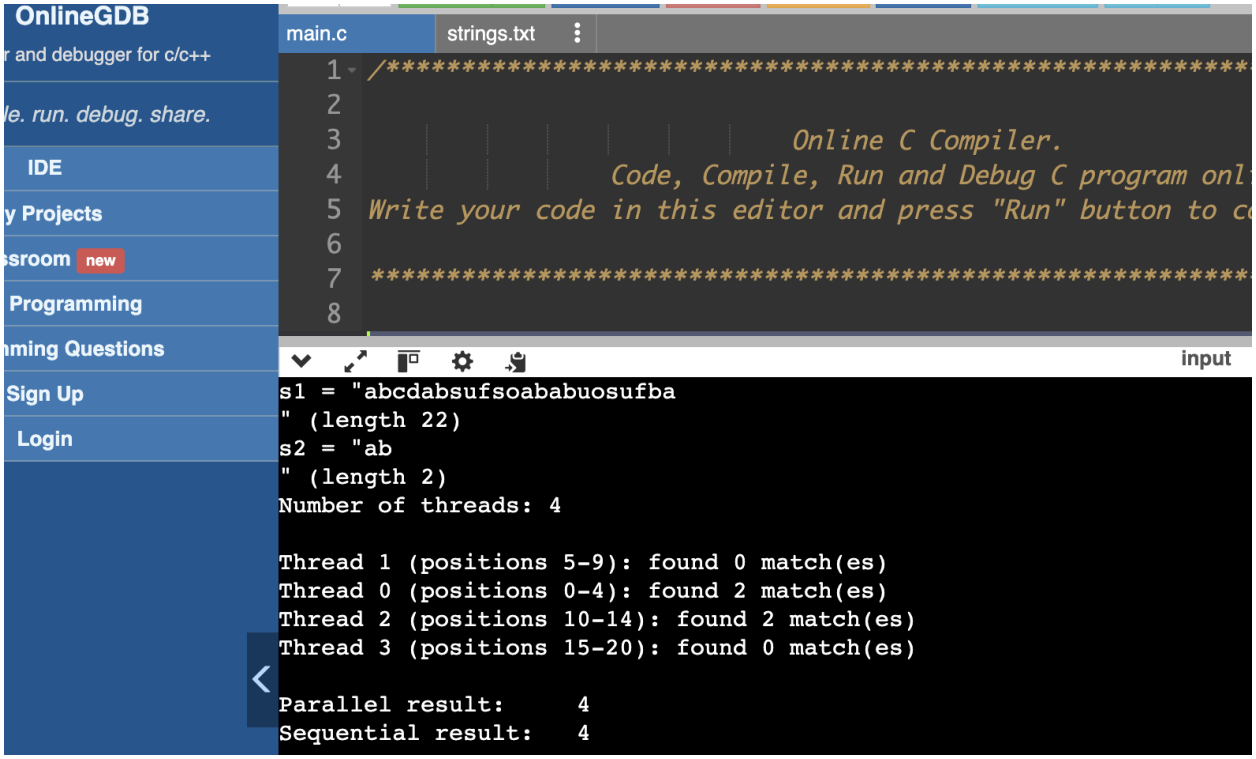
I learned why mutexes are helpful, because my first iteration did not have one, and the total count was incorrect, likely because of race conditions. I also learned a little more about edge cases because I had some issues with other test cases where the substring was being searched against the last chars of the main string for no reason, which made the thread hang at one point

Three interesting facts:
Without a mutex, the pthreads create race conditions to update the total count.
Valid positions to search do not include any chunk shorter than s2 at the end of s1
I also learned that we split up the total work to do no s1 because that would cause overlap and inaccurate results

1.3 How to Run My Code

To run the code, run ./task1 with a strings.txt file or a new strings.txt for further testing

1.4 Output



S1 is set to "abcdabsufsoababuuosufba" length 22
S1 is set to "ab" length 2
4 threads are made and search though the split chunks of 22 0-4, 5-9, 10-14, 15-20 and 21 isnt checked because theres no room for 'ab' in one char
Total count is updated to 4 which matches sequential search

Task 2

2.1 How I Solved the Problems

To solve the consumer producer problem I made 2 threads for cons and prod. These threads have access to a 15 char buffer and then 2 pthread conditions not_full and not_empty. The producer takes chars from message.txt and puts them in the buffer until the buffer is full then it sleeps and lets the consumer print from the buffer to STDOUT until the buffer is empty, this goes until the file read is fully printed.

2.2 What I learned

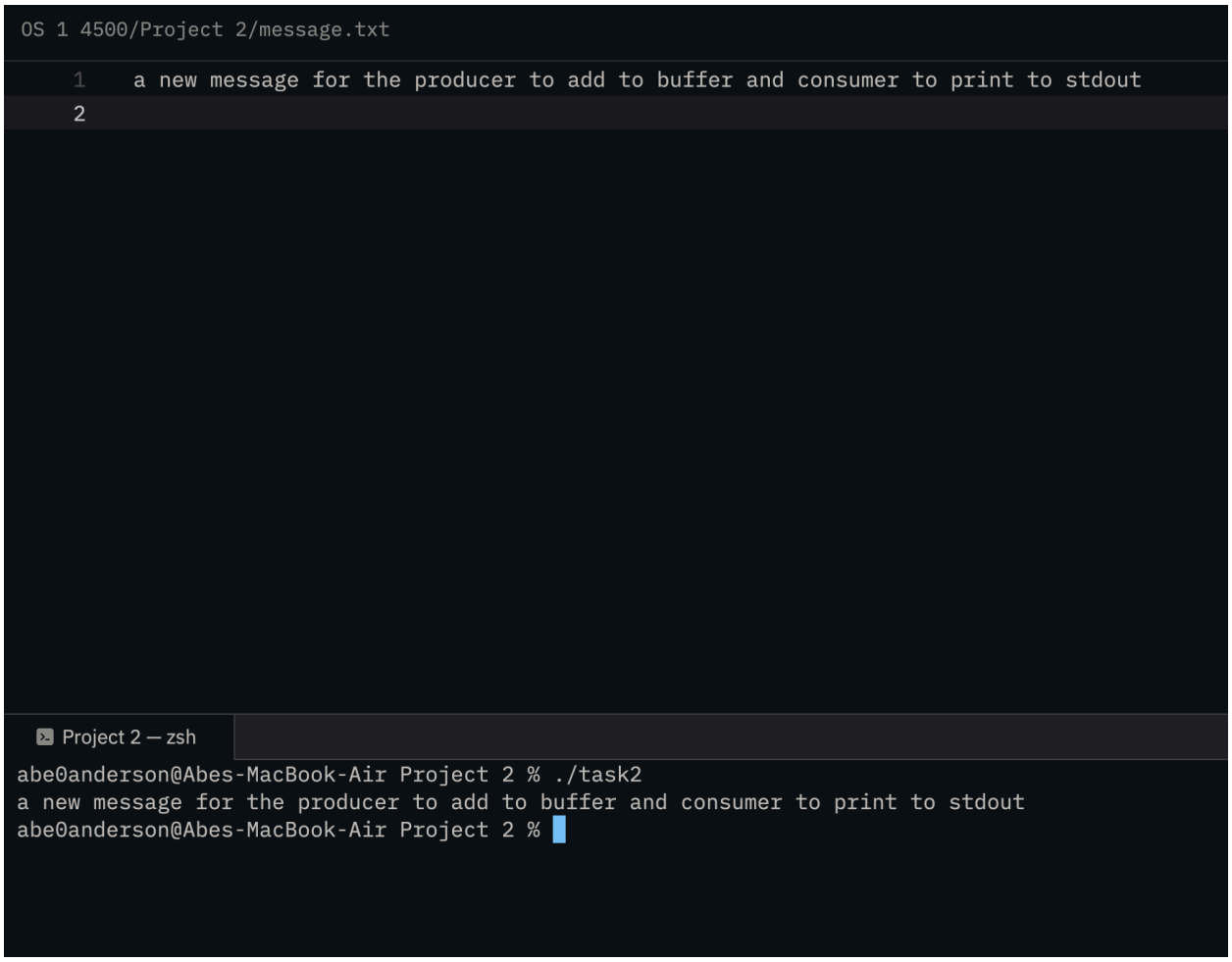
Three things learned:
fflush() can be used to force a print to happen immediately instead of in a memory buffer
while(1) can be used in place of while true

Printing more then one char from a pthread requires a mutex so that multiple threads dont print simultaneously and ruin the message

2.3 How to Run My Code

To run the code, run ./task 2 with a message.txt file

2.4 Code Output Screenshots



task2 prints message from message.txt to console using producer to add chars to 15 char buffer and consumer to print from buffer to STDOUT

Task 3

3.1 How I Solved the Problems

Upon reading the original list_forming.c, I noticed a few things, firstly that using the mutex to lock the global tree for each node update was likely slowing the program down immensely as any other thread that finished and wanted to add its node to the tree would just be blocked waiting its turn. I also noticed the bind_thread_to_cpu was trying to speed up the caching to help speed up the program, but I didnt think this was necessary if the threads just made fewer critical region accesses. To do this each thread creates a local tree with the k nodes then it uses the mutex to access the global tree once to attach the local tree and then it returns. I also noticed that with the batching operations, the cpu binding had little effect so I removed it. I also switched the trylock to lock in order to reduce busy waiting.

3.2 What I learned

I learned that adding the nodes in a already structured tree rather then per node would cause a huge speedup.

I also learned that using lock instead of trylock would be more efficient because lock blocks the thread whereas trylock causes busy waiting which waist cpu cycles.

I also learned that each thread being pinned to a specific cpu made less difference on the batched nodes as oppose to attaching each node from the thread

3.3 How to Run My Code

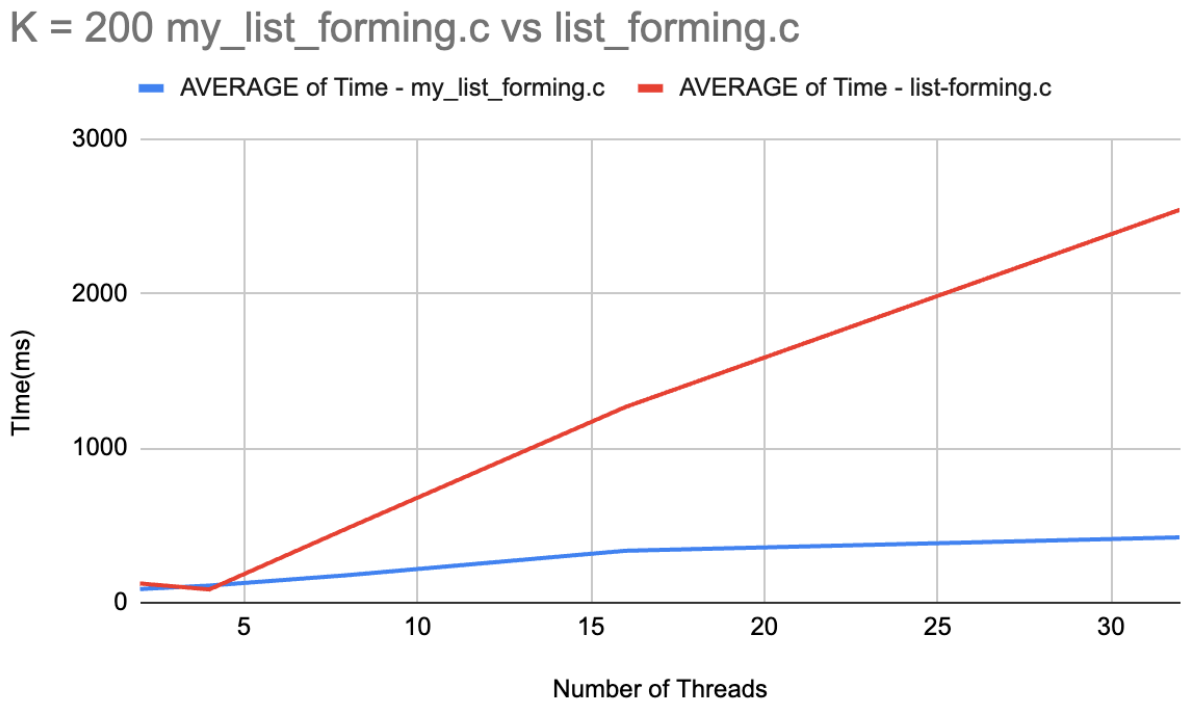
To run my code ./my_list_forming [number of threads]

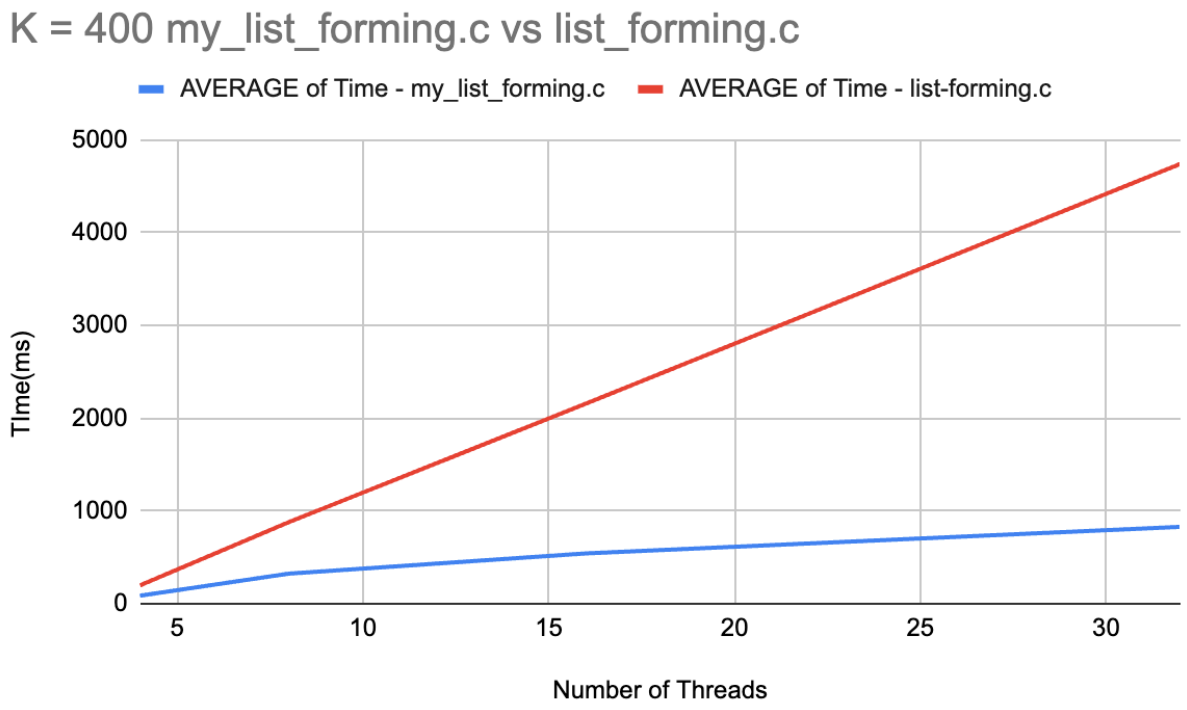
3.4 Output

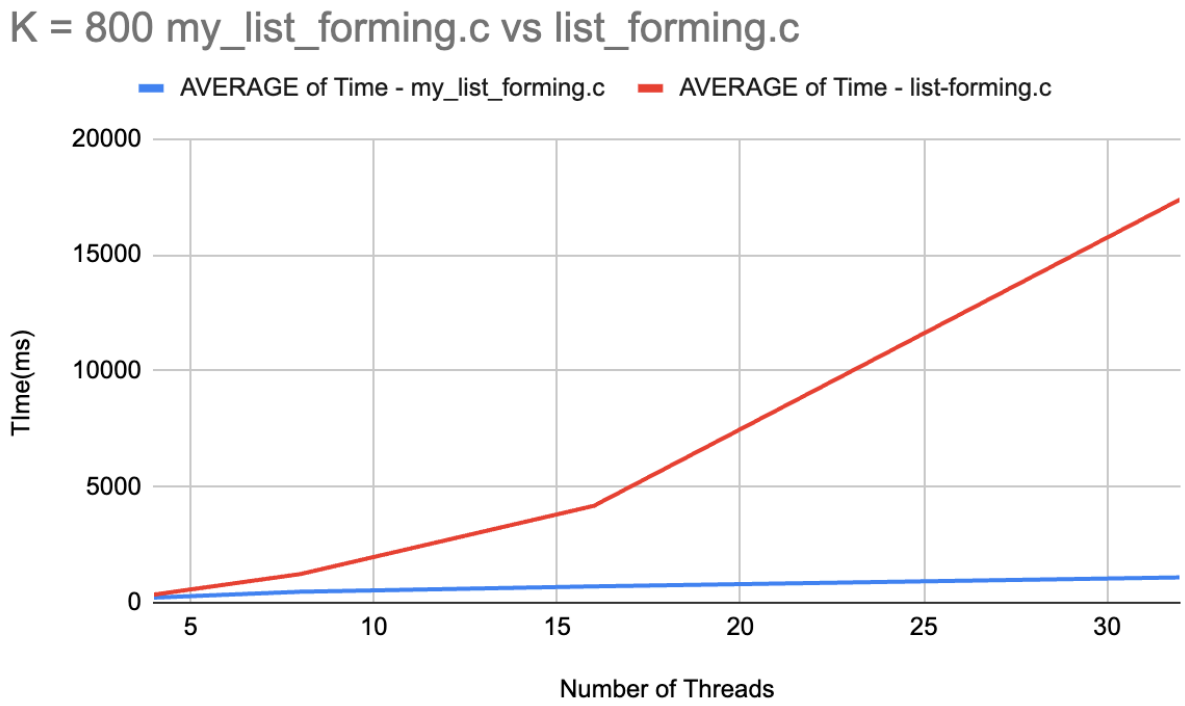
For testing my_list_forming.c I got help from GPT4 to make a script to call it multiple times with the number of threads and K varied and put that result in results.csv which is included in this project zip and was utilized to graph the performance of both c programs. The output of that is good as a test of my program overall cause it shows that it runs and how it scales better with larger k

```
Project 2 — zsh
abe0anderson@Abes-MacBook-Air Project 2 % ./P2Tests.sh
Starting script...
Writing header...
K=200 Threads=2 Run=1 Time=106
K=200 Threads=2 Run=2 Time=74
K=200 Threads=2 Run=3 Time=85
K=200 Threads=4 Run=1 Time=87
K=200 Threads=4 Run=2 Time=118
K=200 Threads=4 Run=3 Time=127
K=200 Threads=8 Run=1 Time=190
K=200 Threads=8 Run=2 Time=156
K=200 Threads=8 Run=3 Time=189
K=200 Threads=16 Run=1 Time=572
K=200 Threads=16 Run=2 Time=200
K=200 Threads=16 Run=3 Time=236
K=200 Threads=32 Run=1 Time=447
K=200 Threads=32 Run=2 Time=437
K=200 Threads=32 Run=3 Time=384
K=400 Threads=2 Run=1 Time=78
K=400 Threads=2 Run=2 Time=69
K=400 Threads=2 Run=3 Time=62
K=400 Threads=4 Run=1 Time=83
K=400 Threads=4 Run=2 Time=96
K=400 Threads=4 Run=3 Time=80
K=400 Threads=8 Run=1 Time=361
K=400 Threads=8 Run=2 Time=360
K=400 Threads=8 Run=3 Time=250
K=400 Threads=16 Run=1 Time=436
K=400 Threads=16 Run=2 Time=703
K=400 Threads=16 Run=3 Time=487
K=400 Threads=32 Run=1 Time=817
K=400 Threads=32 Run=2 Time=785
K=400 Threads=32 Run=3 Time=883
K=800 Threads=2 Run=1 Time=173
K=800 Threads=2 Run=2 Time=232
K=800 Threads=2 Run=3 Time=177
K=800 Threads=4 Run=1 Time=188
K=800 Threads=4 Run=2 Time=143
K=800 Threads=4 Run=3 Time=325
K=800 Threads=8 Run=1 Time=461
K=800 Threads=8 Run=2 Time=492
K=800 Threads=8 Run=3 Time=464
```

And here are graphs of runtime for list-forming.c and my_list_forming.c:


K = 200 my_list_forming.c vs list_forming.c
AVERAGE of Time - my_list_forming.c
AVERAGE of Time - list-forming.c
3000
2000
1000
0
TIme(ms)
5
10
15
20
25
30
Number of Threads


K = 400 my_list_forming.c vs list_forming.c
AVERAGE of Time - my_list_forming.c
AVERAGE of Time - list-forming.c
5000
4000
3000
2000
1000
0
TIme(ms)
5
10
15
20
25
30
Number of Threads



As these show, my_list_forming is much faster accross 1-32 theads for k = 200,400,and 800. This is likely due to less lock contention with the single access to add tree nodes over an access for each node. These show that especially as the number of threads grow my program handles much better.